Purpose

AUD-0 — audit/ Purpose + boundaries

Piece A — Purpose

File: `Axion/libraries/audit/AUD-0_purpose.md`

---
library: audit
id: AUD-0
schema_version: 1.0.0
status: draft

AUD-0 — audit/ Purpose + Boundaries

Purpose
`audit/` defines Axion's **operator action tracking**:
- what actions are recorded (who/what/when/why)
- how actions are written (append-only ledger)
- how actions link to runs, stages, gates, and policy decisions

Audit is for accountability and traceability, not telemetry analytics.

What it governs (in scope)
- Audit event schema (operator actions)
- Audit log/ledger schema (per run / per workspace)
- Required fields for traceability (actor, reason, references)
- Tamper-evident hooks (hash chaining optional)

What it does NOT govern (out of scope)
- Gate evaluation semantics → `gates/`
- Proof ledger semantics → `verification/`
- Telemetry events/metrics → `telemetry/`
- Permissions/authn/authz models → `system/` (audit just records results)

Consumers
- Operator UI (who approved what, overrides, reruns)
- Compliance/audit review
- Policy engine (reads approvals/overrides to justify outcomes)

Determinism requirements
- Audit events are append-only.

- Ordering for serialization is deterministic.
- References are stable ids/paths (not embedded content).

Piece B — Boundary checklist

File: `Axion/libraries/audit/AUD-0_boundary_checklist.md`

---
library: audit
id: AUD-0a
schema_version: 1.0.0
status: draft

AUD-0a — Boundary Checklist

Belongs in `audit/` if it answers:
- “What operator actions are recorded?”
- “How do we prove who approved/overrode something?”
- “How do we link actions to runs/stages/gates/policies?”

Does NOT belong in `audit/` if it answers:
- “What telemetry do we emit?” (telemetry)
- “What proofs exist?” (verification)

Operator action model

AUD-1 — Audit action schema (event types + required fields)

Piece A — Audit action model (concept)

File: `Axion/libraries/audit/AUD-1_audit_action_model.md`

---
library: audit
id: AUD-1
schema_version: 1.0.0
status: draft

AUD-1 — Audit Action Model

Purpose
Define the canonical structure of an audit action so Axion can record:
- who did something
- what they did
- when they did it
- why they did it
- what object/run/stage/gate/policy it affected

What counts as an audit action
Minimum action types:
- run_started
- run_paused
- run_resumed
- run_cancelled
- stage_rerun_requested
- override_requested
- override_approved
- override_denied
- export_requested
- export_approved
- export_denied
- policy_decision_applied
- manual_attestation_recorded

Required fields (minimum)
Every audit action must include:

- audit_event_id
- action_type
- actor
- occurred_at
- target
- reason
- refs

Design rules
- Audit actions are human-accountability records, not telemetry analytics.
- “reason” is required for sensitive actions:
  - overrides
  - reruns
  - exports
  - manual attestations
- References must use stable ids/paths, not embedded content.

Piece B — Audit action schema (JSON Schema)

File: `Axion/libraries/audit/schemas/audit_action.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "audit_action.v1",
  "type": "object",
  "required": [
    "audit_event_id",
    "action_type",
    "actor",
    "occurred_at",
    "target",
    "refs"
  ],
  "properties": {
    "audit_event_id": {
      "type": "string",
      "pattern": "^AUD-[A-Z0-9]{6,}$"
    },

    "action_type": {
      "type": "string",
```

```
  "enum": [
    "run_started",
    "run_paused",
    "run_resumed",
    "run_cancelled",
    "stage_rerun_requested",
    "override_requested",
    "override_approved",
    "override_denied",
    "export_requested",
    "export_approved",
    "export_denied",
    "policy_decision_applied",
    "manual_attestation_recorded"
  ]
},

"actor": {
  "type": "object",
  "required": ["actor_id", "role"],
  "properties": {
    "actor_id": { "type": "string", "minLength": 2 },
    "role": {
      "type": "string",
      "enum": ["owner", "maintainer", "operator", "system"]
    }
  },
  "additionalProperties": false
},

"occurred_at": {
  "type": "string",
  "format": "date-time"
},

"target": {
  "type": "object",
  "required": ["target_type"],
  "properties": {
    "target_type": {
      "type": "string",
      "enum": [
        "run",
        "stage",
```

```
          "gate",
          "override",
          "export",
          "policy_decision",
          "proof"
        ]
      },
      "run_id": { "type": "string" },
      "stage_id": { "type": "string" },
      "gate_id": { "type": "string" },
      "override_request_id": { "type": "string" },
      "override_decision_id": { "type": "string" },
      "policy_decision_id": { "type": "string" },
      "proof_id": { "type": "string" }
    },
    "additionalProperties": false
  },

  "reason": {
    "type": "string",
    "minLength": 3
  },

  "refs": {
    "type": "object",
    "properties": {
      "run_manifest_ref": { "$ref": "#/$defs/ref" },
      "gate_report_ref": { "$ref": "#/$defs/ref" },
      "proof_ref": { "$ref": "#/$defs/ref" },
      "policy_ref": { "$ref": "#/$defs/ref" }
    },
    "additionalProperties": false
  }
},

"$defs": {
  "ref": {
    "type": "object",
    "required": ["path"],
    "properties": {
      "path": { "type": "string" },
      "hash": { "type": "string" }
    },
    "additionalProperties": false
```

```
    }
  },

  "additionalProperties": false
}
```

Piece C — Determinism rules

File: `Axion/libraries/audit/AUD-1_determinism_rules.md`

```
---
library: audit
id: AUD-1a
schema_version: 1.0.0
status: draft
---

# AUD-1a — Determinism Rules (Audit Actions)

- audit_event_id is stable and unique.
- action_type is a closed enum.
- refs use stable ids/paths only.
- Sensitive actions require reason:
  - override_requested
  - override_approved
  - override_denied
  - stage_rerun_requested
  - export_requested
  - export_approved
  - export_denied
  - manual_attestation_recorded
- actor.role is normalized to the closed enum set.
```

Piece D — Validation checklist

File: `Axion/libraries/audit/AUD-1_validation_checklist.md`

```
---
library: audit
id: AUD-1b
```

schema_version: 1.0.0
status: draft

AUD-1b — Validation Checklist

- [ ] audit_action validates against schema
- [ ] action_type is from the allowed enum set
- [ ] actor contains actor_id + normalized role
- [ ] target contains the correct ids for its target_type
- [ ] sensitive actions include a non-empty reason
- [ ] refs contain only stable paths/hashes

Action ledger schema + append-only rules

AUD-2 — Audit log / ledger schema (append-only log)

Piece A — Audit log model (concept)

File: `Axion/libraries/audit/AUD-2_audit_log_model.md`

```
---
library: audit
id: AUD-2
schema_version: 1.0.0
status: draft
---

# AUD-2 — Audit Log / Ledger Model

## Purpose
Define an append-only audit ledger that stores audit_action entries for:
- a run
- a workspace/project scope (optional)

## Ledger properties
- append-only (no deletes/edits without creating a new corrective event)
- deterministic ordering for serialization
- optionally tamper-evident (hash chaining)

## Linkage
Audit actions link to:
- run_id
- stage_id
- gate_id
- policy_decision_id
- proof_id
via the target + refs fields.
```

Piece B — Audit log schema (JSON Schema)

File: `Axion/libraries/audit/schemas/audit_log.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
```

```
  "id": "audit_log.v1",
  "type": "object",
  "required": ["audit_log_id", "scope", "events", "created_at", "updated_at"],
  "properties": {
    "audit_log_id": { "type": "string", "pattern": "^AUDLOG-[A-Z0-9]{6,}$" },

    "scope": {
      "type": "object",
      "required": ["scope_type"],
      "properties": {
        "scope_type": { "type": "string", "enum": ["run", "project", "workspace"] },
        "run_id": { "type": "string" },
        "project_id": { "type": "string" },
        "workspace_id": { "type": "string" }
      },
      "additionalProperties": false
    },

    "events": {
      "type": "array",
      "items": { "$ref": "audit_action.v1.schema.json" }
    },

    "tamper_evident": {
      "type": "object",
      "properties": {
        "enabled": { "type": "boolean" },
        "chain_hash_algo": { "type": "string" },
        "root_hash": { "type": "string" }
      },
      "additionalProperties": false
    },

    "created_at": { "type": "string", "format": "date-time" },
    "updated_at": { "type": "string", "format": "date-time" }
  },
  "additionalProperties": false
}
```

Piece C — Tamper-evident hash chaining (optional spec)

File: `Axion/libraries/audit/AUD-2_tamper_evident_rules.md`

---
library: audit
id: AUD-2a
schema_version: 1.0.0
status: draft

AUD-2a — Tamper-evident Rules (Optional)

Concept
Each event stores/derives a chain hash:
hash_i = H(hash_{i-1} + canonical_json(event_i))

Where:
- H is a pinned algorithm (e.g., sha256)
- canonical_json uses stable key ordering and no whitespace variance

Storage options
Option 1: store chain hashes alongside events (extra field)
Option 2: compute on demand and store only root_hash

Minimum requirement
If enabled:
- chain_hash_algo must be set and pinned
- root_hash must match the computed final hash

Piece D — Determinism rules (log serialization)

File: `Axion/libraries/audit/AUD-2_determinism_rules.md`

---
library: audit
id: AUD-2b
schema_version: 1.0.0
status: draft

AUD-2b — Determinism Rules (Audit Log)

- events are append-only.
- serialization order is deterministic:
  - occurred_at

  - audit_event_id
- If tamper_evident enabled:
  - canonical JSON form must be used for hashing
  - hash algorithm is pinned.

Piece E — Validation checklist

File: `Axion/libraries/audit/AUD-2_validation_checklist.md`

---
library: audit
id: AUD-2c
schema_version: 1.0.0
status: draft

AUD-2c — Validation Checklist

- [ ] audit_log validates against schema
- [ ] scope fields match scope_type (run/project/workspace)
- [ ] events validate against audit_action schema
- [ ] events behave append-only
- [ ] deterministic ordering applied for serialization
- [ ] tamper-evident: root_hash matches computed (if enabled)

Audit index + query keys

AUD-3 — Audit index + query keys

Piece A — Audit index model (concept)

File: `Axion/libraries/audit/AUD-3_audit_index_model.md`

```
---
library: audit
id: AUD-3
schema_version: 1.0.0
status: draft
---

# AUD-3 — Audit Index + Query Keys

## Purpose
Provide a deterministic index so the system and Operator UI can query audit history without scanning entire ledgers.

## Index responsibilities
- map audit_log scopes → storage locations
- provide query keys for fast filtering:
 - by run_id
 - by stage_id
 - by gate_id
 - by action_type
 - by actor_id
 - by policy_decision_id / override_request_id / proof_id
 - by time window (occurred_at)

## Design rules
- The index is a registry of references, not a copy of events.
- The index must never contain sensitive payloads; only ids/refs.
```

Piece B — Audit index schema

File: `Axion/libraries/audit/schemas/audit_index.v1.schema.json`

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "schema_version": "1.0.0",
 "id": "audit_index.v1",
```

```
"type": "object",
"required": ["index_id", "schema_version", "entries", "updated_at"],
"properties": {
  "index_id": { "type": "string", "pattern": "^AUDINDEX-[A-Z0-9]{6,}$" },
  "schema_version": { "type": "string", "minLength": 1 },
  "updated_at": { "type": "string", "format": "date-time" },

  "entries": {
    "type": "array",
    "items": { "$ref": "#/$defs/entry" }
  }
},
"$defs": {
  "entry": {
    "type": "object",
    "required": ["scope", "audit_log_ref", "query_keys"],
    "properties": {
      "scope": {
        "type": "object",
        "required": ["scope_type"],
        "properties": {
          "scope_type": { "type": "string", "enum": ["run", "project", "workspace"] },
          "run_id": { "type": "string" },
          "project_id": { "type": "string" },
          "workspace_id": { "type": "string" }
        },
        "additionalProperties": false
      },

      "audit_log_ref": {
        "type": "object",
        "required": ["path"],
        "properties": {
          "path": { "type": "string" },
          "hash": { "type": "string" }
        },
        "additionalProperties": false
      },

      "query_keys": {
        "type": "object",
        "properties": {
          "run_id": { "type": "string" },
          "stage_ids": { "type": "array", "items": { "type": "string" } },
```

```
          "gate_ids": { "type": "array", "items": { "type": "string" } },
          "action_types": { "type": "array", "items": { "type": "string" } },
          "actor_ids": { "type": "array", "items": { "type": "string" } },
          "policy_decision_ids": { "type": "array", "items": { "type": "string" } }
        },
        "additionalProperties": false
      }
    },
    "additionalProperties": false
  }
},
"additionalProperties": false
}
```

Piece C — Determinism rules (indexing)

File: `Axion/libraries/audit/AUD-3_determinism_rules.md`

```
---
library: audit
id: AUD-3a
schema_version: 1.0.0
status: draft
---

# AUD-3a — Determinism Rules (Audit Index)

- Index stores references only (no event payloads).
- query_keys are derived deterministically from the ledger contents:
 - unique sets, sorted lexicographically
- entries are sorted deterministically:
 - scope_type, then run_id/project_id/workspace_id
- audit_log_ref.hash optional; if present it must match ledger file hash.
```

Piece D — Validation checklist

File: `Axion/libraries/audit/AUD-3_validation_checklist.md`

```
---
library: audit
id: AUD-3b
schema_version: 1.0.0
```

status: draft

AUD-3b — Validation Checklist (Audit Index)

- [ ] audit_index validates against schema
- [ ] entries contain valid scope + audit_log_ref
- [ ] query_keys contain only ids (no sensitive content)
- [ ] deterministic ordering of entries + key sets

Integrity rules

AUD-4 — Integrity rules (tamper-evidence, hashing if used)

Piece A — Integrity model (concept)

File: `Axion/libraries/audit/AUD-4_integrity_model.md`

```
---
library: audit
id: AUD-4
schema_version: 1.0.0
status: draft
---

# AUD-4 — Integrity Rules (Tamper-Evidence)

## Purpose
Define how audit records can be trusted as unchanged after recording.

## Integrity goals
- detect silent edits to audit events
- detect deleted/reordered events
- support optional tamper-evident verification without changing audit semantics

## Integrity levels
### Level 0 — Append-only only
- events are append-only
- no hash chain required

### Level 1 — File hash
- the serialized audit log has a recorded hash
- detects file changes but not per-event history structure

### Level 2 — Hash chain
- each event contributes to a rolling chain hash
- detects edits, deletions, and reordering

## Design rule
Tamper-evidence is optional but strongly recommended for:
- compliance runs
- override-heavy workflows
- external exports
```

Piece B — Integrity registry (machine-readable)

File: `Axion/libraries/audit/registries/audit_integrity.v1.json`

```
{
 "schema_version": "1.0.0",
 "registry_id": "audit_integrity.v1",
 "integrity_mode": "hash_chain",
 "hash_algorithm": "sha256",
 "canonical_json": true,
 "required_for_risk_classes": {
   "PROTOTYPE": "file_hash",
   "PROD": "hash_chain",
   "COMPLIANCE": "hash_chain"
 }
}
```

Piece C — Hash chain rules

File: `Axion/libraries/audit/AUD-4_hash_chain_rules.md`

```
---
library: audit
id: AUD-4a
schema_version: 1.0.0
status: draft
---

# AUD-4a — Hash Chain Rules

## Chain formula
For event_i:

chain_hash_i = H(chain_hash_{i-1} + canonical_json(event_i))

Where:
- H = pinned hash algorithm from audit_integrity registry
- canonical_json(event_i) uses stable key ordering and normalized whitespace
- chain_hash_0 = fixed genesis value, e.g. "AUDIT_GENESIS_V1"

## What this detects
```

- event edits
- event deletion
- event insertion in the middle
- event reordering

Storage choices
You may:
1) store per-event chain hashes, or
2) store only the final root hash and recompute when verifying

Recommended:
- store root_hash in audit_log
- optionally store per-event chain_hash in derived verification output, not inline

---

Piece D — Determinism rules

File: `Axion/libraries/audit/AUD-4_determinism_rules.md`

---
library: audit
id: AUD-4b
schema_version: 1.0.0
status: draft

AUD-4b — Determinism Rules (Integrity)

- Integrity mode is pinned by audit_integrity registry.
- Canonical JSON serialization is required for any hashing mode.
- Event ordering for hash computation is:
  - occurred_at
  - audit_event_id
- Genesis value is fixed and versioned.
- If hash_algorithm changes, registry version must bump.

Piece E — Validation checklist

File: `Axion/libraries/audit/AUD-4_validation_checklist.md`

---
library: audit
id: AUD-4c

schema_version: 1.0.0
status: draft

AUD-4c — Validation Checklist (Integrity)

- [ ] audit_integrity registry exists and is pinned
- [ ] integrity_mode is one of: append_only, file_hash, hash_chain
- [ ] hash_algorithm is explicitly set when hashing is used
- [ ] canonical JSON serialization rules are defined
- [ ] root_hash / file hash verification succeeds when enabled
- [ ] required integrity level is enforced by risk class

Gates

AUD-5 — Gates (schema valid, monotonic timestamps, actor present)

Piece A — Audit gates intent

File: `Axion/libraries/audit/AUD-5_audit_gates.md`

---
library: audit
id: AUD-5
schema_version: 1.0.0
status: draft

AUD-5 — Audit Gates

Purpose
Ensure audit data is structurally valid, chronologically sane, and accountable.

Gate set (minimum)

AUD-GATE-01 — Audit log schema valid
- AUDIT_LOG exists
- validates against audit_log.v1
- all events validate against audit_action.v1

AUD-GATE-02 — Actor present
- every audit event has actor.actor_id
- every audit event has actor.role from closed enum
- no anonymous sensitive actions

AUD-GATE-03 — Monotonic timestamps
- occurred_at timestamps are monotonic in ledger order
- no event timestamp is earlier than a prior event in the same serialized order

AUD-GATE-04 — Target/reference coherence
- target fields match target_type
- refs point to valid paths/ids where required
- override/export/rerun events carry the required related ids

AUD-GATE-05 — Sensitive actions include reason
- overrides, exports, reruns, and manual attestations require non-empty reason

AUD-GATE-06 — Integrity mode satisfied
- if integrity mode is file_hash or hash_chain:
  - integrity verification passes
- required integrity level must meet risk-class minimum

Piece B — Machine-readable gate spec (optional)

File: `Axion/libraries/audit/AUD-5_audit_gates.spec.json`

```
{
  "schema_version": "1.0.0",
  "contract_id": "AUD-5a",
  "gates": [
    {
      "gate_id": "AUD-GATE-01",
      "name": "Audit log schema valid",
      "severity": "hard_stop",
      "checks": [
        "AUDIT_LOG exists",
        "schema_valid(audit_log.v1)",
        "all events schema_valid(audit_action.v1)"
      ]
    },
    {
      "gate_id": "AUD-GATE-02",
      "name": "Actor present",
      "severity": "hard_stop",
      "checks": [
        "every event has actor.actor_id",
        "every event has actor.role"
      ]
    },
    {
      "gate_id": "AUD-GATE-03",
      "name": "Monotonic timestamps",
      "severity": "hard_stop",
      "checks": [
        "occurred_at monotonic in serialized event order"
      ]
    },
    {
```

```
    "gate_id": "AUD-GATE-04",
    "name": "Target/reference coherence",
    "severity": "hard_stop",
    "checks": [
      "target fields match target_type",
      "required refs present for sensitive actions"
    ]
  },
  {
    "gate_id": "AUD-GATE-05",
    "name": "Sensitive actions include reason",
    "severity": "hard_stop",
    "checks": [
      "overrides require reason",
      "exports require reason",
      "reruns require reason",
      "manual attestations require reason"
    ]
  },
  {
    "gate_id": "AUD-GATE-06",
    "name": "Integrity mode satisfied",
    "severity": "policy_controlled",
    "checks": [
      "if integrity enabled then integrity verification passes",
      "risk-class minimum integrity level satisfied"
    ]
  }
 ]
}
```

Piece C — Evidence requirements

File: `Axion/libraries/audit/AUD-5_evidence_requirements.md`

---
library: audit
id: AUD-5a
schema_version: 1.0.0
status: draft

AUD-5a — Evidence Requirements (Audit Failures)

On failure, evidence must include:
- pointer to failing audit log
- failing event ids
- missing actor/reason fields
- timestamp ordering violations:
  - prior event id + timestamp
  - current event id + timestamp
- target/ref mismatch details
- integrity verification result (expected vs actual hash/root_hash)

Remediation:
- repair audit generation logic
- re-record missing required fields through corrective event if allowed
- rerun integrity verification
- block sensitive operation until audit requirements are satisfied

---

Piece D — Determinism rules

File: `Axion/libraries/audit/AUD-5_determinism_rules.md`

---
library: audit
id: AUD-5b
schema_version: 1.0.0
status: draft

AUD-5b — Determinism Rules (Audit Gates)

- Gates run in fixed order AUD-GATE-01..06.
- Event order for evaluation is the ledger serialization order:
  - occurred_at
  - audit_event_id
- Missing/failed event lists are sorted deterministically by audit_event_id.
- Integrity verification uses pinned audit_integrity registry and canonical JSON rules.

Piece E — Validation checklist

File: `Axion/libraries/audit/AUD-5_validation_checklist.md`

---
library: audit
id: AUD-5c
schema_version: 1.0.0
status: draft

AUD-5c — Validation Checklist

- [ ] Audit gates cover schema validity, actor presence, monotonic timestamps, target/ref coherence, required reasons, and integrity verification
- [ ] Event ordering for evaluation is fixed and deterministic
- [ ] Evidence pointers + remediation are included
- [ ] Integrity level enforcement can vary by risk class/policy

Ops workflow

AUD-6 — Ops workflow (retention, redaction, export)

Piece A — Ops workflow model (concept)

File: `Axion/libraries/audit/AUD-6_ops_workflow.md`

---
library: audit
id: AUD-6
schema_version: 1.0.0
status: draft

AUD-6 — Ops Workflow (Retention, Redaction, Export)

Purpose
Define how audit data is handled operationally after it is recorded:
- how long it is retained
- what must be redacted before sharing
- how exports are packaged and approved

Audit data must remain useful for traceability without leaking sensitive information.

Workflow surfaces
1) **Retention**
- how long audit logs are kept
- what gets archived vs deleted
- risk-class-specific retention expectations

2) **Redaction**
- what fields must never leave internal scope
- what fields can be masked or removed
- how exported audit views differ from internal audit logs

3) **Export**
- when audit logs may be exported
- who can approve export
- what export format is allowed

Piece B — Retention + export policy registry

File: `Axion/libraries/audit/registries/audit_ops_policy.v1.json`

```
{
 "schema_version": "1.0.0",
 "policy_id": "AUDOPS-BASE01",
 "retention": {
   "PROTOTYPE": {
     "active_days": 30,
     "archive_days": 180
   },
   "PROD": {
     "active_days": 90,
     "archive_days": 365
   },
   "COMPLIANCE": {
     "active_days": 365,
     "archive_days": 2555
   }
 },
 "redaction": {
   "deny_keys": [
     "actor_email",
     "actor_ip",
     "session",
     "token",
     "authorization",
     "cookie"
   ],
   "mask_actor_ids_on_external_export": false,
   "strip_internal_refs_on_external_export": true
 },
 "export": {
   "allowed_formats": ["json"],
   "external_export_requires_approval": true,
   "include_hashes": true
 }
}
```

Piece C — Redaction + export rules

File: `Axion/libraries/audit/AUD-6_redaction_export_rules.md`

---

library: audit
id: AUD-6a
schema_version: 1.0.0
status: draft

AUD-6a — Redaction + Export Rules

Internal view
Internal audit logs may include:
- actor_id
- role
- target ids
- refs to internal artifacts
But still must not include secrets/tokens.

External export
External audit export must:
- strip deny_keys
- remove internal-only refs if policy says so
- include only fields needed for traceability:
 - audit_event_id
 - action_type
 - actor.role
 - occurred_at
 - target ids
 - reason (if safe)
- preserve integrity evidence if possible:
 - root_hash / file hash
 - hash algorithm

Retention workflow
- Active period: available for normal querying
- Archive period: read-only, slower access allowed
- End of retention: delete or purge per policy and legal requirements

---

Piece D — Determinism rules

File: `Axion/libraries/audit/AUD-6_determinism_rules.md`

---
library: audit
id: AUD-6b

schema_version: 1.0.0
status: draft

AUD-6b — Determinism Rules (Ops Workflow)

- audit_ops_policy registry is pinned.
- Retention class is derived from run risk class.
- Redaction/export behavior is deterministic:
 - drop deny_keys
 - strip internal refs when configured
 - preserve event ordering
- External export does not modify source audit log; it produces a derived export artifact.

Piece E — Validation checklist

File: `Axion/libraries/audit/AUD-6_validation_checklist.md`

---
library: audit
id: AUD-6c
schema_version: 1.0.0
status: draft

AUD-6c — Validation Checklist

- [ ] audit_ops_policy registry exists and validates
- [ ] retention windows defined for PROTOTYPE, PROD, COMPLIANCE
- [ ] deny_keys defined for redaction
- [ ] external export approval requirement explicit
- [ ] export behavior is deterministic and non-destructive

Minimum viable audit

AUD-7 — Minimum viable audit set (runtime-required files + “done”)

Piece A — MV audit inventory

File: `Axion/libraries/audit/AUD-7_minimum_viable_set.md`

---
library: audit
id: AUD-7
schema_version: 1.0.0
status: draft

AUD-7 — Minimum Viable audit/ Set

Goal
Define the smallest audit library set required to record accountable operator actions with deterministic integrity and operational handling.

Required runtime files (minimum)
1) Core docs
- AUD-0_purpose.md
- AUD-0_boundary_checklist.md
- AUD-1_audit_action_model.md
- AUD-1_determinism_rules.md
- AUD-1_validation_checklist.md
- AUD-2_audit_log_model.md
- AUD-2_tamper_evident_rules.md
- AUD-2_determinism_rules.md
- AUD-2_validation_checklist.md
- AUD-3_audit_index_model.md
- AUD-3_determinism_rules.md
- AUD-3_validation_checklist.md
- AUD-4_integrity_model.md
- AUD-4_hash_chain_rules.md
- AUD-4_determinism_rules.md
- AUD-4_validation_checklist.md
- AUD-5_audit_gates.md
- AUD-5_evidence_requirements.md
- AUD-5_determinism_rules.md
- AUD-5_validation_checklist.md

- AUD-6_ops_workflow.md
- AUD-6_redaction_export_rules.md
- AUD-6_determinism_rules.md
- AUD-6_validation_checklist.md
- AUD-7_minimum_viable_set.md

2) Schemas (must be in schema_registry)
- schemas/audit_action.v1.schema.json
- schemas/audit_log.v1.schema.json
- schemas/audit_index.v1.schema.json

3) Registries (runtime defaults)
- registries/audit_integrity.v1.json
- registries/audit_ops_policy.v1.json

4) Gate specs (optional)
- AUD-5_audit_gates.spec.json

Piece B — Definition of done (audit/)

File: `Axion/libraries/audit/AUD-7_definition_of_done.md`

---
library: audit
id: AUD-7a
schema_version: 1.0.0
status: draft

AUD-7a — audit/ Definition of Done

audit/ is “done” when:

Schemas + registries
- [ ] All audit schemas validate (JSON Schema check)
- [ ] audit_integrity registry exists and is pinned
- [ ] audit_ops_policy registry exists and is pinned

Runtime behavior (contract-level)
- [ ] Sensitive operator actions are recorded as audit actions
- [ ] Audit log is append-only and serializes deterministically
- [ ] Audit index supports query keys without storing sensitive payloads
- [ ] Integrity verification works for the configured mode

- [ ] Retention/redaction/export workflow is defined and deterministic

Gates
- [ ] AUD-GATE-01..06 implemented
- [ ] Missing actor/reason/timestamp coherence failures are detected
- [ ] Sensitive actions cannot proceed without required audit records

Safety
- [ ] External audit exports are redacted per policy
- [ ] Internal logs do not include secrets/tokens

Piece C — Minimal folder tree

File: `Axion/libraries/audit/AUD-7_minimal_tree.txt`

```
Axion/libraries/audit/
 AUD-0_purpose.md
 AUD-0_boundary_checklist.md
 AUD-1_audit_action_model.md
 AUD-1_determinism_rules.md
 AUD-1_validation_checklist.md
 AUD-2_audit_log_model.md
 AUD-2_tamper_evident_rules.md
 AUD-2_determinism_rules.md
 AUD-2_validation_checklist.md
 AUD-3_audit_index_model.md
 AUD-3_determinism_rules.md
 AUD-3_validation_checklist.md
 AUD-4_integrity_model.md
 AUD-4_hash_chain_rules.md
 AUD-4_determinism_rules.md
 AUD-4_validation_checklist.md
 AUD-5_audit_gates.md
 AUD-5_audit_gates.spec.json
 AUD-5_evidence_requirements.md
 AUD-5_determinism_rules.md
 AUD-5_validation_checklist.md
 AUD-6_ops_workflow.md
 AUD-6_redaction_export_rules.md
 AUD-6_determinism_rules.md
 AUD-6_validation_checklist.md
 AUD-7_minimum_viable_set.md
 AUD-7_definition_of_done.md
```

AUD-7_minimal_tree.txt

```
schemas/
  audit_action.v1.schema.json
  audit_log.v1.schema.json
  audit_index.v1.schema.json

registries/
  audit_integrity.v1.json
  audit_ops_policy.v1.json
```